



BAAI
智源研究院



北京大学
PEKING UNIVERSITY

AI Compiler: Basics and Optimizations

Review: Programming with Triton



More examples...

- FlagGems (to register with the ATen backend of PyTorch)
 - https://github.com/flagos-ai/FlagGems/tree/master/src/flag_gems/ops
- Triton kernels in SGLang
 - <https://github.com/sgl-project/sglang/tree/main/python/sglang/kernels/ops>



BAAI
智源研究院



北京大学
PEKING UNIVERSITY

- **General Flow**
- AI-DSL Design
- Optimization Passes

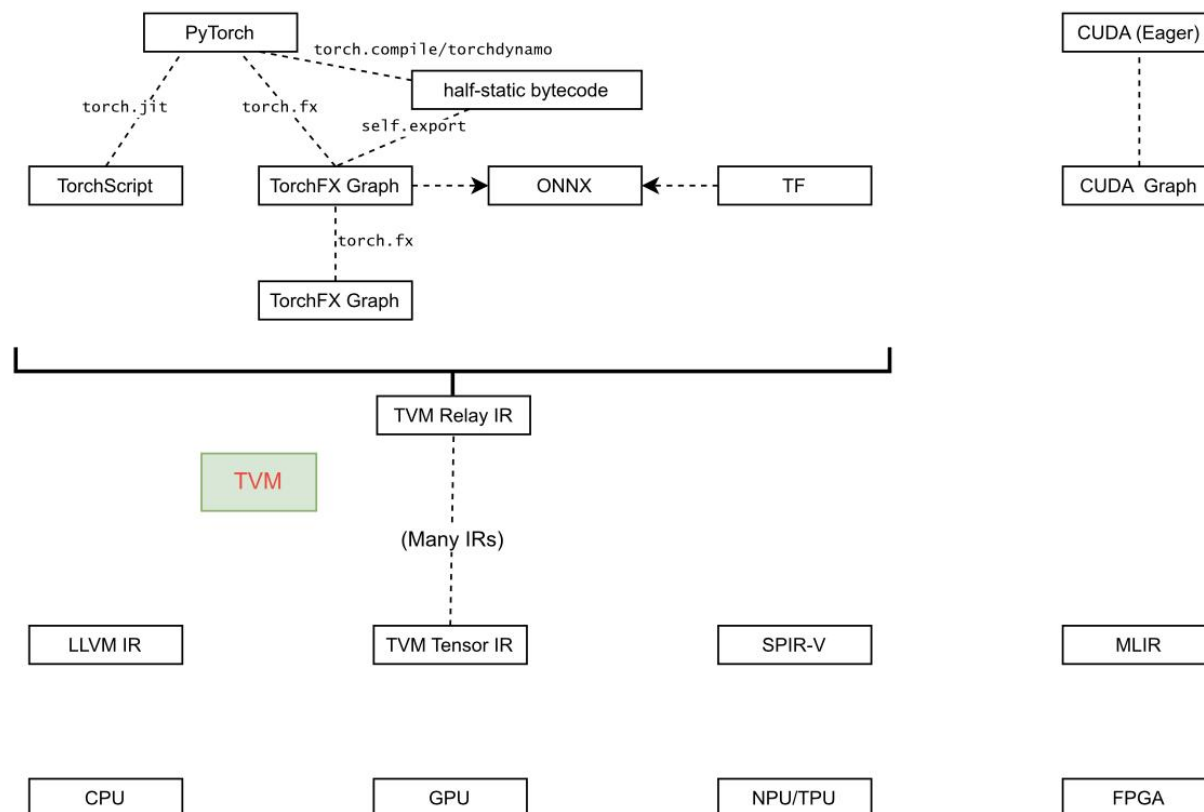
General Flow: Compilation Procedure



- High-level representation lowering to hardware code gradually
 - High-level framework: pyTorch, TensorFlow, ...
 - High-level DSL: Triton, Tilelang, ...
 - Low-level DSL: CUDA, Ascend C, ...
 - Intermediate Representations: MLIR (TIR, TGIR) , LLVM, ...
 - Hardware Code: PTX, CPU ISA, NPU ISA, ...
- Introduce hardware information gradually and optimize automatically
 - Computation graph optimization, parameter sweeping, register assignment, ...
- IR/DSL to meet the needs of AI computation
 - TVM: tensor operation
 - Tilelang: tile-level abstract

General Flow: Computation Graph

- Computation graph: Design the dependency among operators in AI-computing tasks
 - Static graph (Tensorflow1.x)
 - Specify at compilation, easy to optimize
 - Dynamic graph (PyTorch)
 - Specify at runtime, fit for modern LLM
- Why computation graph
 - Represent forward and backward relationship
 - Fit for multi-level AI operator representation
- Components
 - Nodes: Tensor/Data or operation
 - Edges: Operation or Tensor/Data



General Flow: Computation Graph



BAAI
智源研究院



北京大学
PEKING UNIVERSITY

- Computation graph construction
 - Static: Define variable and operation before compilation->Obtain computation graph
 - Dynamic

- Define variables
- Forward pass
- Construct on the fly
- Backward pass

- Optimization time

- Static: compilation
- Dynamic: runtime

```
# Importing torch
import torch

# Initializing input tensors
a = torch.tensor(15.0, requires_grad=True)
b = torch.tensor(20.0, requires_grad=True)

# Computing the output
c = a * b

# Computing the gradients
c.backward()

# Collecting the output gradient of the
# output with respect to the input 'a'
derivative_out_a = a.grad

# Collecting the output gradient of the
# output with respect to the input 'b'
derivative_out_b = b.grad

# Displaying the outputs
print(f'c = {c}')
print(f'Derivative of c with respect to a = {derivative_out_a}')
print(f'Derivative of c with respect to b = {derivative_out_b}')
```

PyTorch Dynamic Graph

```
# Importing tensorflow version 1
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()

# Initializing placeholder variables of
# the graph
a = tf.placeholder(tf.float32)
b = tf.placeholder(tf.float32)

# Defining the operation
c = tf.multiply(a, b)

# Instantiating a tensorflow session
with tf.Session() as sess:

    # Computing the output of the graph by giving
    # respective input values
    out = sess.run([c], feed_dict={a: [15.0], b: [20.0]})[0][0]

    # Computing the output gradient of the output with
    # respect to the input 'a'
    derivative_out_a = sess.run(tf.gradients(c, a), feed_dict={
        a: [15.0], b: [20.0]})[0][0]

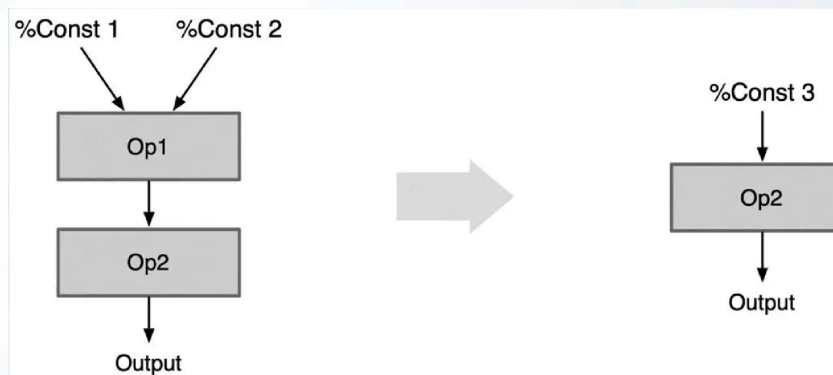
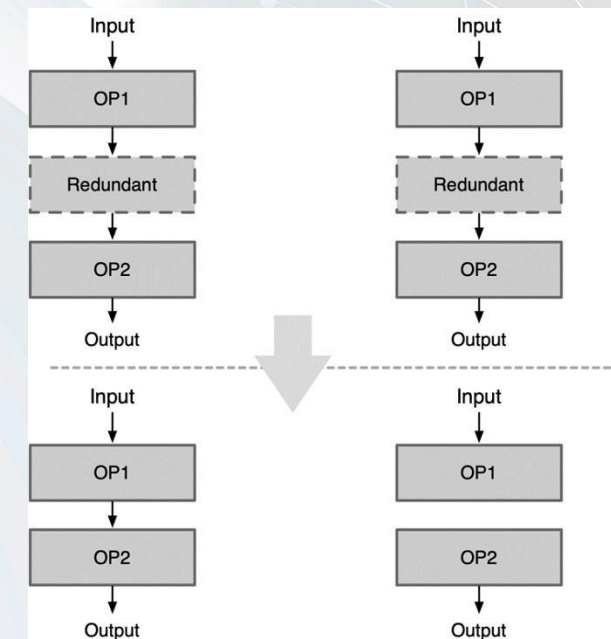
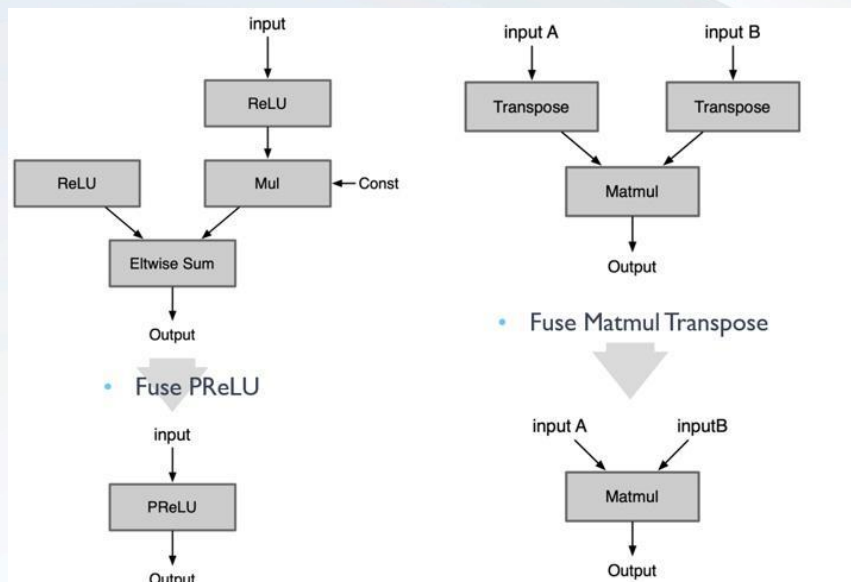
    # Computing the output gradient of the output with
    # respect to the input 'b'
    derivative_out_b = sess.run(tf.gradients(c, b), feed_dict={
        a: [15.0], b: [20.0]})[0][0]

    # Displaying the outputs
    print(f'c = {out}')
    print(f'Derivative of c with respect to a = {derivative_out_a}')
    print(f'Derivative of c with respect to b = {derivative_out_b}')
```

Tensorflow Static Graph

General Flow: Graph Optimization

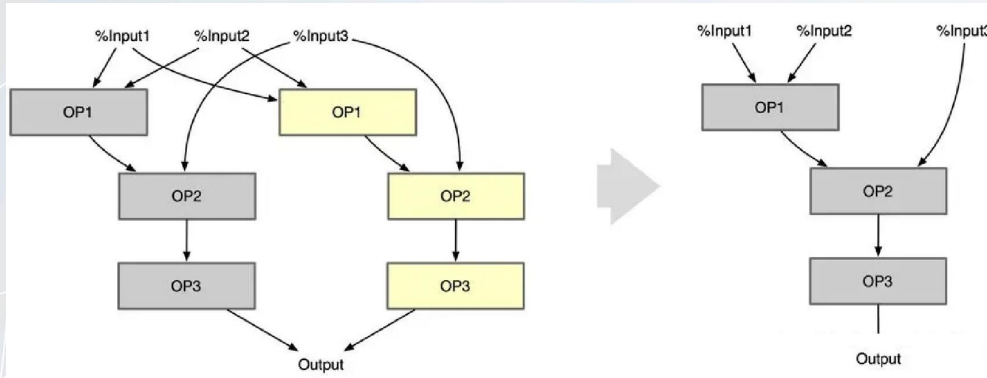
- Operation fusion
 - Reduce data transmission
 - Reduce launch overhead
 - Simplify graph pattern
- Redundant node elimination
 - Certain operations are inactive
 - cast between same data types
- Constant folding
 - Calculate in advance for constants



General Flow: Graph Optimization



- Common subgraph optimization
 - Common subgraph: same structure, parameter and input
 - Reuse subgraph

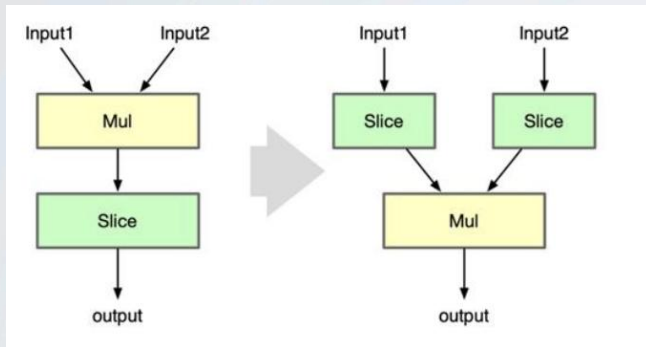


- Operator substitute
 - Equivalent operator substitution for certain hardware feature
 - Conv can be calculated by TensorCore by adding some additional calculation
 - Two consecutive conv can be substitute by an equivalent conv

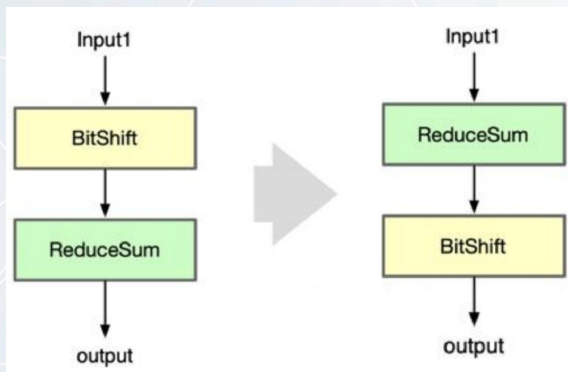
General Flow: Graph Optimization



- Operation scheduling
 - Calculate some operation in advance to reduce duplicate computation



- Reduce data transfer cost



- **Dymanic-Static Tradeoff**
 - Both PyTorch and TensorFlow extends thier computation graph (torch.fx and TensorFlow2.0)
 - Easy to develop and efficiently optimized for execution
- **torch.compile: Benchmark and Optimization On-the-fly**
 - Main idea: Optimize current computation as long as it does not change
 - Faster Compilation and faster execution after first run

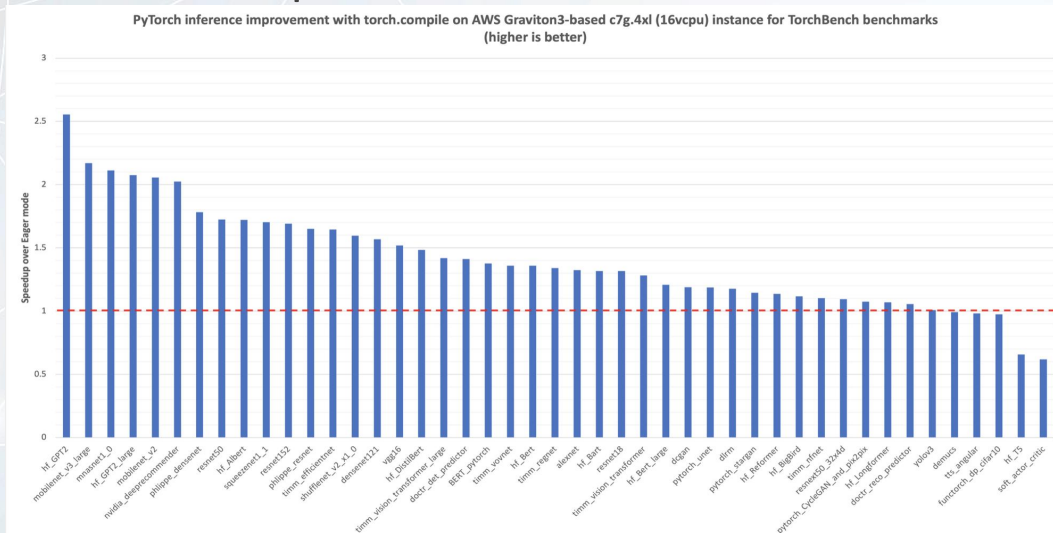
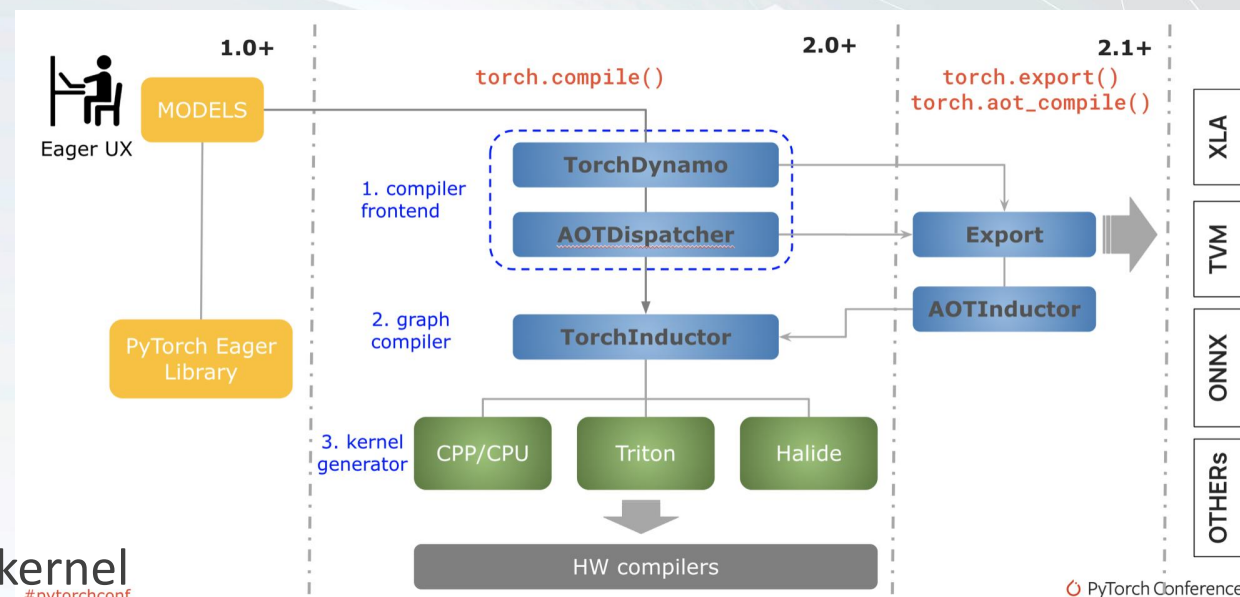


Image Source: <https://pytorch.ac.cn/blog/accelerated-pytorch-inference/>

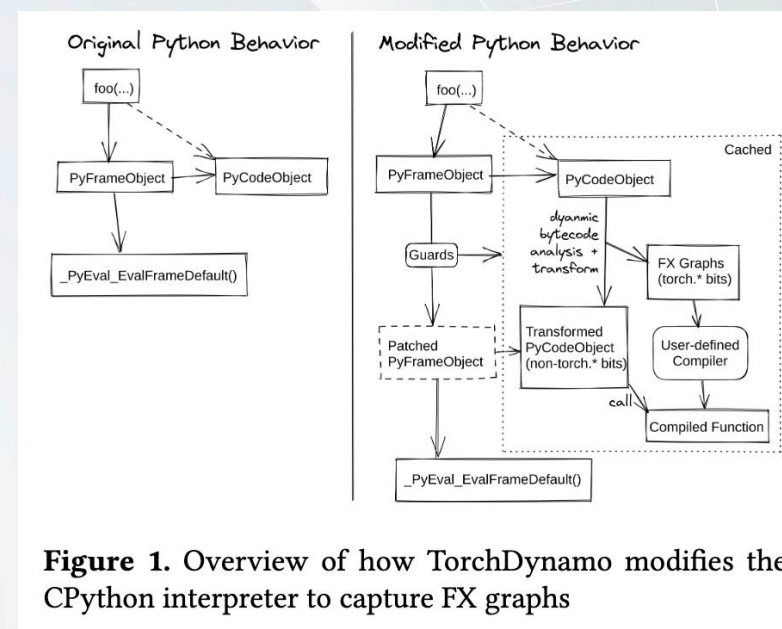
General Flow: torch.compile

- **TorchDynamo**
 - Rewrite python bytecode
 - Retrieved computation graph from bytecode
 - Output forward static graph in first run
- **AOTAutoGrad**
 - AOTDispatcher: dispatch operator to specific kernel
 - AOT-ahead of time: trace BP graph simultaneously as forward computation graph
- **TorchInductor**
 - Optimize graph and generate hardware code
 - Leverage kernel fusion and other methods for specific architecture



General Flow: TorchDynamo Overview

- Eager Mode Pros and Cons:
 - PyTorch define-by-run mechanism is highly flexible and well-loved by researchers. However, the framework can only see one operator at a time, making cross-operator graph-level optimization (e.g., operator fusion, memory scheduling) difficult.
- Compromises of Previous Approaches:
 - TorchScript: Attempts static parsing of Python AST, facing “all-or-nothing” dilemma, easily failing because it doesn't support complete Python semantics.
 - Lazy Tensors: High runtime overhead, delaying low-level kernel launch and causing frequent recompilation.
- TorchDynamo Core Motivation:
 - As a Python-level Just-In-Time compiler, the goal is to achieve Python graph compilation without sacrificing any PyTorch flexibility.



General Flow: TorchDynamo Example

An Example

```
from typing import List
import torch
import torchdynamo
```

```
def toy_example(a, b):
    x = a / (torch.abs(a) + 1)
    if b.sum() < 0:
        b = b * -1
    return x * b
```

```
def my_compiler(gm: torch.fx.GraphModule,
                example_inputs: List[torch.Tensor]):
    print("my_compiler() called with FX graph:")
    gm.graph.print_tabular()
    return gm.forward # return a python callable
```

```
with torchdynamo.optimize(my_compiler):
    for _ in range(100):
        toy_example(torch.randn(10),
                    torch.randn(10))
```

Example Output

```
my_compiler() called with FX graph:
opcode      name      target      args
-----
placeholder a         a           ()
placeholder b         b           ()
call_function abs_1      torch.abs   (a,)
call_function add      operator.add (abs_1, 1)
call_function truediv   operator.truediv (a, add)
call_method  sum_1     sum         (b,)
call_function lt        operator.lt  (sum_1, 0)
output      output    output      ((truediv, lt),)
```

```
my_compiler() called with FX graph:
opcode      name      target      args
-----
placeholder b         b           ()
placeholder x         x           ()
call_function mul      operator.mul (b, -1)
call_function mul_1     operator.mul (x, mul)
output      output    output      ((mul_1,),)
```

```
my_compiler() called with FX graph:
opcode      name      target      args
-----
placeholder b         b           ()
placeholder x         x           ()
call_function mul      operator.mul (x, b)
output      output    output      ((mul,),)
```

General Flow: TorchInductor



- Graph fusion: Combines multiple pointwise operators
 - May generate equivalent Triton kernels on the fly
- Scheduling: Determines loop ordering, tile sizes, and parallel execution
- Backend Selection
 - GPU → Triton
 - CPU → C++/OpenMP
- Memory planning: reduces temporary tensor allocation

- General Flow
- **AI-DSL Design**
- Optimization Passes

AI-DSL Design: Why DSL?



- Gap between AI computing abstract and GPU execution model
 - Deep learning models are naturally expressed as tensor algebra
 - GPUs execute massively parallel scalar instructions
- Human/Agent workload
 - Explicit thread/block management, Memory hierarchy (global, shared, registers), Synchronization primitives, Instruction scheduling, Architecture-specific optimizations, ...
- DSLs separate algorithm description from performance implementation
- Modern AI compilers increasingly rely on domain-specific IRs

AI-DSL Design: How to Design DSL



- Tensor-Centric programming model
 - Matrix multiplication, Convolution, Reduction, Broadcast, Elementwise computation
- Declarative semantics
 - describe mathematical operations rather than execution order
 - $C = A @ B$
 - $y = \text{softmax}(x)$
- Hardware Independence
 - NVIDIA GPU, AMD GPU, CPU, TPU, Future accelerators
- Compiler optimizations enabled by DSLs
 - Operator fusion, Layout transformation, Automatic tiling, Parallel scheduling

AI-DSL Design: Triton

- More friendly GPU development DSL
- Compiler Flow
 - Dialect->Triton IR->LLVM IR->PTX

Compare with CUDA

	CUDA	TRITON
Memory Coalescing	Manual	Automatic
Shared Memory Management	Manual	Automatic
Scheduling (Within SMs)	Manual	Automatic
Scheduling (Across SMs)	Manual	Manual

Optimization effect (softmax)

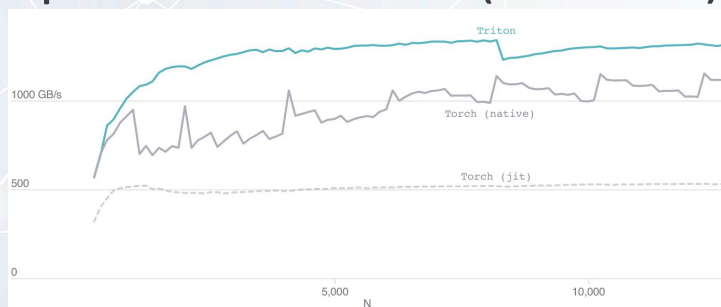


Image source: <https://openai.com/index/triton/>

```
@jit
def add(X, Y, Z, N):
    pid = program_id(0)
    idx = pid * 512 + arange(512)
    mask = idx < N
    x = load(X + idx, mask=mask)
    y = load(Y + idx, mask=mask)
    store(Z + idx, x + y, mask=mask)
```



```
def void add(i32* X .aligned(16) , i32* Y .aligned(16) , i32*
Z .aligned(16) , i32 N .multipleof(2) ) {
```

```
entry:
    %0 = get_program_id[0] i32;
    %1 = mul i32 %0, 512;
    %3 = make_range[0 : 512] i32<512>;
    %4 = splat i32<512> %1;
    %6 = add i32<512> %4, %3;
    %9 = splat i32<512> N;
    %11 = icmp_slt i1<512> %6, %9;
    %14 = splat i32*<512> X;
    %16 = getelementptr i32*<512> %14, %6;
    %19 = broadcast i1<512> %11;
    %21 = splat i32<512> undef;
    %22 = masked_load i32<512> %16, %19, %21;
    %26 = splat i32*<512> Y;
    %28 = getelementptr i32*<512> %26, %6;
    %31 = broadcast i1<512> %11;
    %33 = splat i32<512> undef;
    %34 = masked_load i32<512> %28, %31, %33;
    %38 = splat i32*<512> Z;
    %40 = getelementptr i32*<512> %38, %6;
    %43 = add i32<512> %22, %34;
    %46 = broadcast i32<512> %43;
    %48 = broadcast i1<512> %11;
    masked_store void %40, %46, %48;
    ret void;}
```

```
visible .entry add(
    .param .u64 add_param_0, .param .u64 add_param_1,
    .param .u64 add_param_2, .param .u32 add_param_3
)
.maxntid 128, 1, 1
{
    .reg .pred    %p<4>;
    .reg .b32    %r<18>;
    .reg .b64    %rd<8>;
    ld.param.u64    %rd4, [add_param_0];
    ld.param.u64    %rd5, [add_param_1];
    mov.u32    %r13, %tid.x;
    ld.param.u32    %r14, [add_param_3];
    shl.b32    %r15, %r13, 2;
    mov.u32    %r16, %ctaid.x;
    mad.lo.s32    %r17, %r16, 512, %r15;
    setp.ge.s32    %p3, %r17, %r14;
    setp.lt.s32    %p1, %r17, %r14;
    mul.wide.s32    %rd7, %r17, 4;
    add.s64    %rd2, %rd4, %rd7;
    @%p1 ld.global.cg.v4.b32 {%r5,%r6,%r7,%r8}, [ %rd2 + 0];
    add.s64    %rd3, %rd5, %rd7;
    @%p1 ld.global.cg.v4.b32 {%r9,%r10,%r11,%r12}, [ %rd3 + 0];
    @%p3 bra    LBB0_2;
    ld.param.u64    %rd6, [add_param_2];
    add.s64    %rd1, %rd6, %rd7;
    add.s32    %r1, %r5, %r9;
    add.s32    %r2, %r6, %r10;
    add.s32    %r3, %r7, %r11;
    add.s32    %r4, %r8, %r12;
    st.global.v4.u32    [%rd1], {%r1, %r2, %r3, %r4};
LBB0_2:
    ret;
}
```

AI-DSL Design: Triton

- Expose block-parallelism
 - Each kernel instance carry on block operation in parallel

BLOCK = 512

This is a GPU kernel in Triton.
 # Different instances of this
 # function may run in parallel.
 @jit

```
def add(X, Y, Z, N):
    pid = program_id(0)
    # block of indices
    idx = pid * BLOCK + arange(BLOCK)
    mask = idx < N
    ...
```

Triton uses pointer arithmetics
 # rather than indexing operators
 x = load(X + idx, mask=mask)
 y = load(Y + idx, mask=mask)
 store(Z + idx, x + y, mask=mask)

```
...
grid = (ceil_div(N, BLOCK),)
# no thread-block
add[grid](x, y, z, x.shape[0])
```

- Allocate shared memory based on variable living analysis

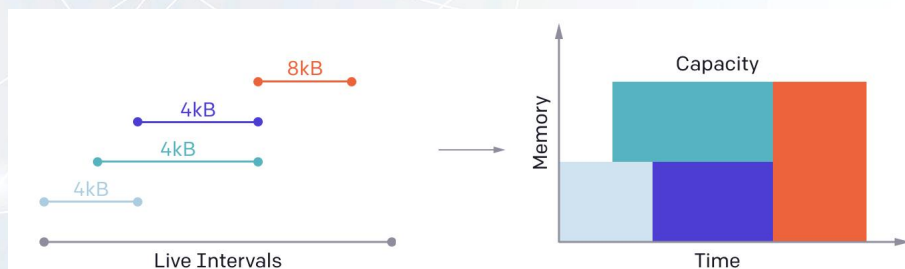
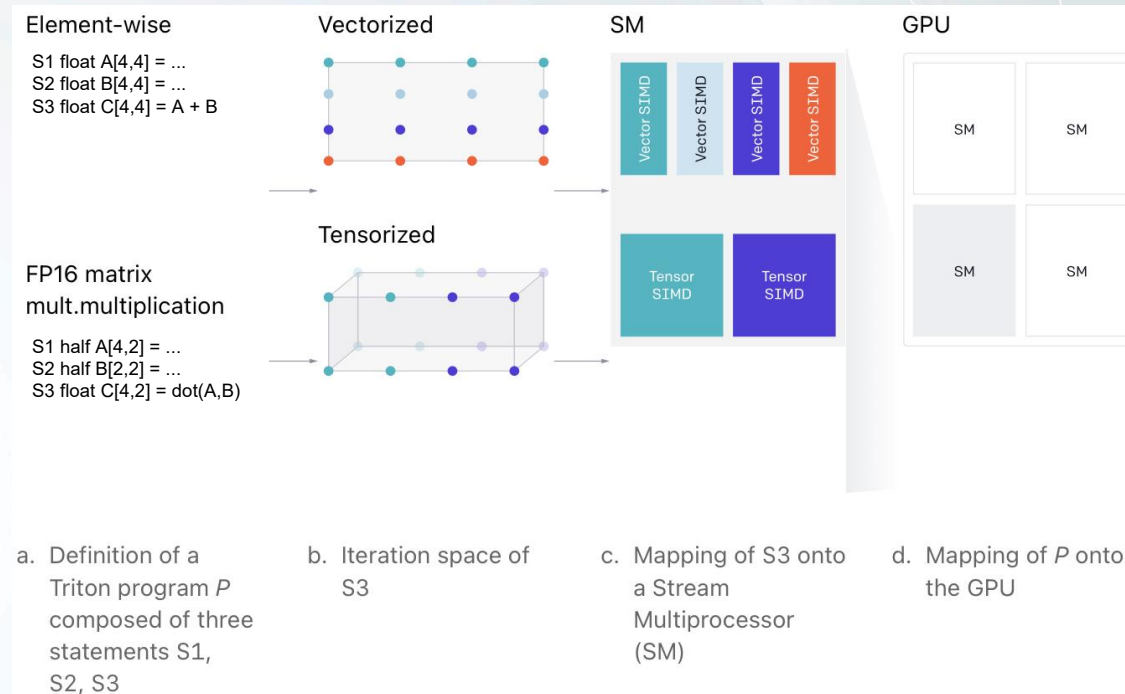


Image source: <https://openai.com/index/triton/>

- automatically parallelize operations both by instance and by SMs
 - Analyze the iteration space of each operation then partition



AI-DSL Design: Triton

- Autotuning

- Consider tile sizes
- Explore at compilation

```
@triton.autotune(configs=[
    triton.Config(kwarg={'BLOCK_SIZE': 128}, num_warps=4),
    triton.Config(kwarg={'BLOCK_SIZE': 1024}, num_warps=8),
],
key=['x_size'] # the two above configs will be evaluated anytime
                # the value of x_size changes
)
@triton.jit
def kernel(x_ptr, x_size, **META):
    BLOCK_SIZE = META['BLOCK_SIZE']
```

- Optimization passes

- Computation graph pattern substitution
- Dead code elimination
- ...

- Use MLIR to lower layer by layer

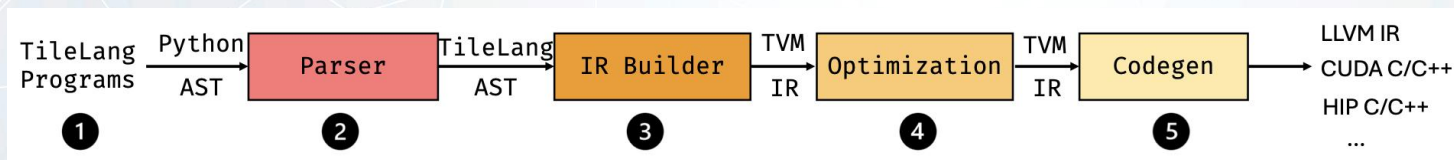
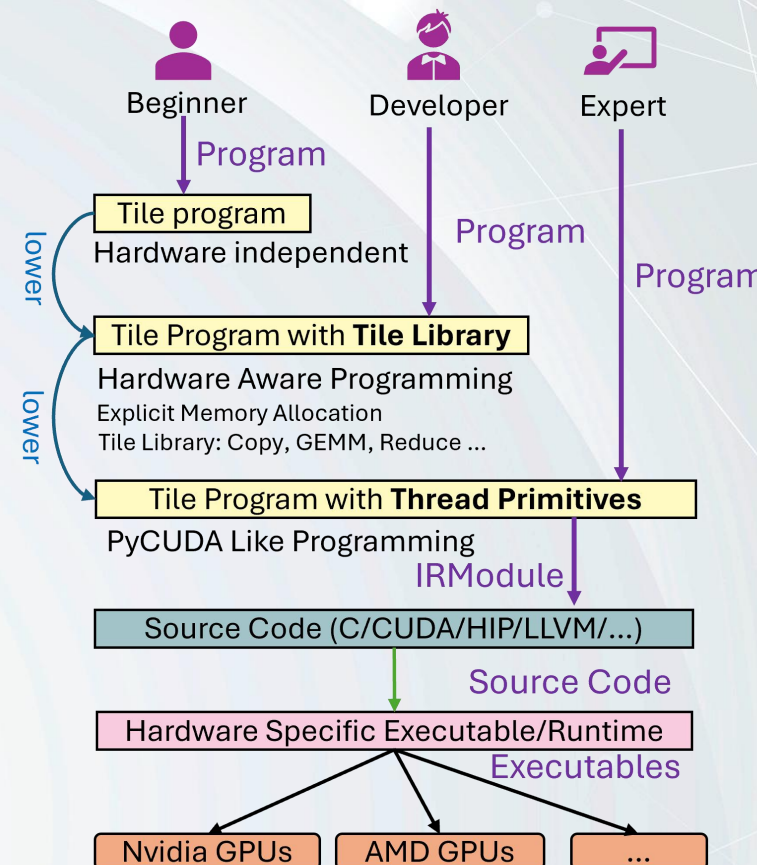
- Most passes reuse MLIR

- Arith: addf, addi, constant, cmpf, cmpi, ...
- Math: exp, sin, cos, log, absi, absf, sqrt
- StructuredControlFlow: for, if, while, yield, condition
- Reduce cost lowering to LLVM

- LLVM emits final GPU code

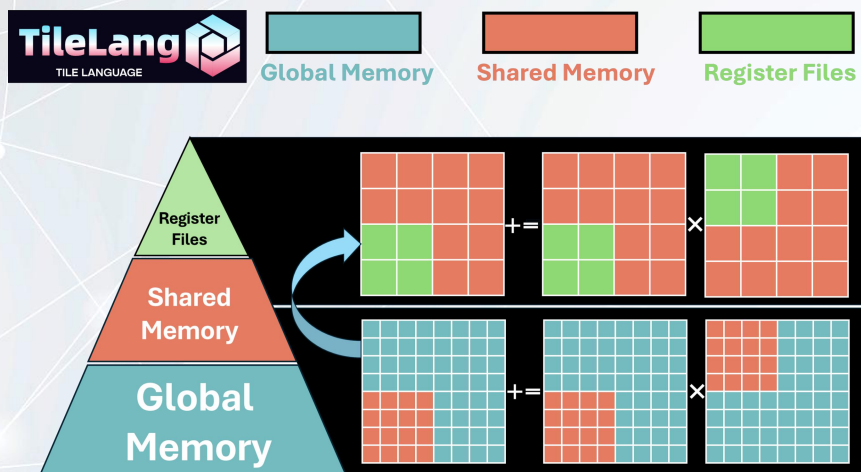
AI-DSL Design: TileLang

- Provide three different programming interfaces
 - Beginner Level: Hardware-Unaware
 - Developer Level: Hardware-Aware with Tile Library
 - Expert Level: Hardware-Aware with Thread Primitives
- Compilation Flow
 - Tile Program: A high-level specification of the computation
 - Tile Program with Tile Library
 - Tile Program with Thread Primitives
 - IRModule
 - Source Code Generation (C/CUDA/HIP/LLVM/...)



AI-DSL Design: TileLang

- Tile declarations: Tiles as First-Class Objects
 - A tile represents a shaped portion of data
 - Can be owned and manipulated by a warp, thread block, or equivalent parallel unit
 - A and B buffers are read in tiled chunks (in example below)
 - Execution context: thread block index (bx and by) and the number of threads
 - Make it easier for the TileLang to automatically optimize



(a) Efficient GEMM with Multi-Level Tiling on GPUs

```
import tilelang.language as T

def Matmul(A: T.Buffer, B: T.Buffer, C: T.Buffer):
    with T.Kernel(
        ceildiv(N, block_N), ceildiv(M, block_M), threads=128
    ) as (bx, by):
        # Kernel Context Initialization

        # Buffer Allocation
        A_shared = T.alloc_shared((block_M, block_K))
        B_shared = T.alloc_shared((block_K, block_N))
        C_local = T.alloc_fragment((block_M, block_N), accum_dtype)
        T.clear(C_local)  # Initialize Accumulate Buffer with Zero

        # Main Loop with Pipeline Annotation
        for k in T.Pipelined(ceildiv(K, block_K), num_stages=3):
            # Copy Data from Global to Shared Memory
            T.copy(A[by * block_M, k * block_K], A_shared)
            T.copy(B[k * block_K, bx * block_N], B_shared)

            # GEMM
            T.gemm(A_shared, B_shared, C_local)

        # Write Back to Global Memory
        T.copy(C_local, C[by * block_M, bx * block_N])
```

(b) Describing Tiled GPU GEMM with TileLang

AI-DSL: TileLang

- Explicit Hardware Memory Allocation:
 - Exposes user-facing intrinsics for tile buffers placement in the hardware memory hierarchy
 - Allocates memory in on-chip storage space
 - T.alloc_shared
 - Corresponds to shared memory on NVIDIA GPUs
 - Allocates accumulators in fragment memory
 - T.alloc_fragment
 - corresponds to register files on NVIDIA GPUs

- Explicit Hardware Memory Allocation:
 - T.clear: Initialize hardware-specific buffers
 - T.fill: Almost same as T.clear
 - T.copy: Data transfer between global memory and hardware-specific memory
 - T.Parallel: operations can also be performed in parallel for data assignments

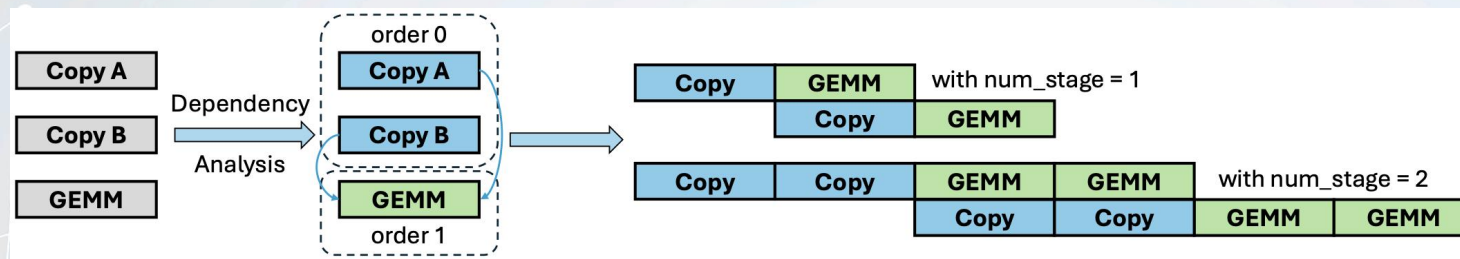
- Abstracts a Set of Tile Operators for Users to Use or Define

	class GEMM	class Copy	class Parallel
class TileOperator { Stmt Lower(); LayoutMap InferLayout(); }	T.GEMM(A_shared, B_shared, C_local) T.call_extern("cute::gemm_ss", args) AShared: MakeSwizzleLayout, BShared: MakeSwizzleLayout, CLocal: MakeMMASTMatrixLayout,	T.Copy(A, A_shared) for i in T.vectorized(8): A_shared[tx, i] = A[tx, i] Loop Layout Lambda i, j->(tid, local_id)	for i, j in T.Parallel(128, 8): C_local[i, j] *= Scale[j] for i in T.vectorized(8): C_local[tx, i] += Scale[i] Scale: InferredScaleLayout,

- Each Tile Operator is required to implement two key interfaces
 - Lower: Defines how the high-level Tile Operator is lowered into a lower-level IR
 - InferLayout: Determine the memory and loop layouts associated with the Tile Operator
- Several Pre-defined Operators
 - gemm: highly optimized implementation for general matrix multiplication
 - reduce: flexible and efficient reduction mechanism for aggregating data across dimensions
 - atomic: atomic operations for safe updates to shared or global memory in a parallel context

TileLang: Schedule Annotations and Primitives

- TileLang provides a comprehensive suite of scheduling primitives
- T.Pipelined: allows efficient pipelined execution of loops to improve performance
 - Overlapping computation and memory operations

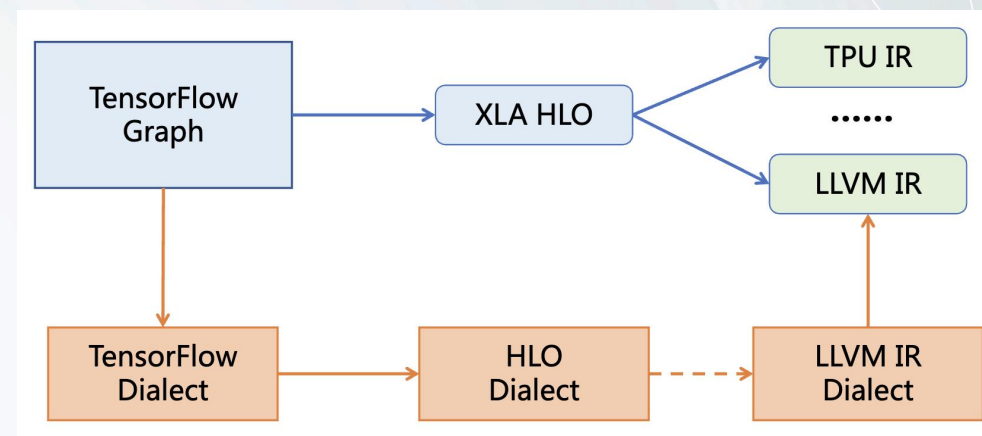


- T.Parallel: automatic parallelization of loops by mapping iterations to threads
- T.annotate_layout: specify memory layout optimizations for shared or global memory
- T.use_swizzle: Improves L2 cache locality by enabling swizzled memory accesses

- General Flow
- AI-DSL Design
- **Optimization Passes**

Optimization Passes: MLIR Architecture

- MLIR A reusable, extensible open-source compilation infrastructure framework
 - Addressing software fragmentation
 - Support for compilation targeting heterogeneous hardware
 - Reducing overhead for domain-specific compiler development
 - Connecting existing various compilers
- MLIR Why Can Solve the Problem
 - Unified syntax, reusable components
 - Shared ecosystem, coordinated optimization across levels
 - Small gaps between levels, easy to implement



Optimization Passes: MLIR Components



- **Module:**
 - Top-level container encapsulating the entire compilation unit, isolating external variable scopes .
- **Region:**
 - Breaking through flat CFG limitations, preserving loops as “first-class citizens” in nested structures.
- **Block:**
 - IntroducingBlock Arguments avoiding traditional LLVM high-maintenance PHI nodes
- **Operation and Attribute:**
 - Highly unified opaque API, along with globally unique attribute dictionaries for representing constants.

```
%results:2 = "d.operation"(%arg0, %arg1) ({
  // Regions belong to Ops and can have multiple blocks.
  ^block(%argument: !d.type):
    // Ops have function types (expressing mapping).
    %value = "nested.operation"() ({
      // Ops can contain nested regions.
      "d.op"() : () -> ()
    }) : () -> (!d.other_type)
    "consume.value"(%value) : (!d.other_type) -> ()
  ^other_block:
    "d.terminator"() [^block(%argument : !d.type)] : () -> ()
})
// Ops can have a list of attributes.
{attribute="value" : !d.type} : () -> (!d.type, !d.other_type)
```

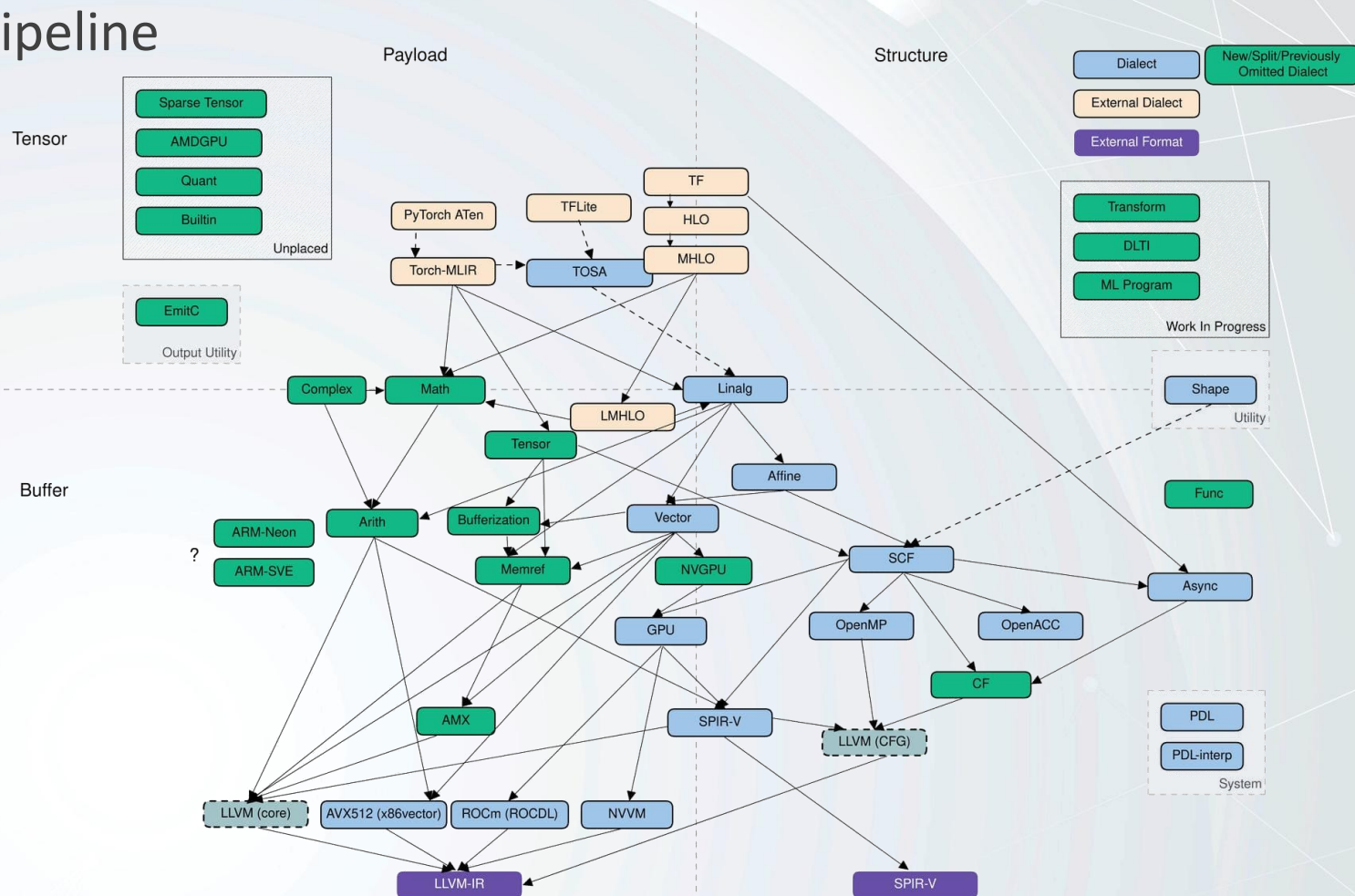
MLIR Introduction

- MLIR: A Reusable Compilation Pipeline

- Reusable IR、 Pass
- Serialize/De-serialize System
- Multiple dialect can co-exist

- MLIR Dialect Translation Graph

- Payload: Actual Operations (+-*//)
- Structure: Control Flow; Function
- Tensor: high level tensors
- Buffer: low level pointers



Optimization Passes: TTIR



- Triton lowering process
 - Python Triton Kernel
 - Triton Frontend (AST + JIT specialization)
 - TTIR (Triton MLIR Dialect)
 - TTGIR (Triton GPU Dialect)
 - LLVM IR (NVVM Dialect)
 - PTX
 - SASS
- Tensor-Level optimization passes (TTIR)
- Canonicalization: simplifies IR by
 - constant folding
 - algebraic simplification
 - dead operation removal
 - redundant cast elimination
- Layout Propagation
 - blocked layouts
 - shared layouts
 - sliced layouts

Optimization Passes: TTGIR

- GPU Mapping Passes
 - Lowering Tensor Programs onto GPU Execution Model
- TTGIR introduces GPU hardware concepts absent from TTIR
- Thread Mapping
 - Maps tensor tiles onto, programs, warps, lanes
- Shared Memory Promotion
 - Frequently reused tensors
 - Global memory->shared memory

- Software Pipelining
 - overlap memory latency
 - controlled by num_stages
 - Reschedule can help enhance efficiency

Load(k+1) Compute(k) Store(k-1)

- Block Tiling
 - MxN matrix -> BLOCK_M × BLOCK_N tiles
 - Specified/autotuned by python triton kernel

Optimization Passes: TTGIR

- Warp-level optimization
 - warp shuffle
 - warp reduction
 - warp synchronization
- Memory coalescing
 - Transforms scattered accesses into contiguous memory transactions
- Vectorization
 - improving memory bandwidth
 - load f32->load.v4.f32
- Tensor core pattern recognition
 - Detects matrix multiply patterns
 - lower to mma.sync

Thread
0 -> A0
1 -> A64
2 -> A3

Thread
0 -> A0
1 -> A1
2 -> A2

Optimization Passes: LLVM

- LLVM no longer sees tensors
 - SSA values
 - basic blocks
 - scalar instructions
- mem2reg
 - Promotes stack variables into registers
- Instruction Combining
 - $(a+b)+c \rightarrow a+(b+c)$
 - $\text{mul}(x,2) \rightarrow \text{shift}$
- Global Value Numbering
 - Eliminates redundant computations across control flow
- Loop-Invariant code motion
 - Moves invariant computations outside loops
- Register allocation
 - Assigns virtual registers to physical GPU registers
- LLVM performs classical compiler optimizations independent of machine learning

Optimization Passes: Parameter and Effect

- Relating kernel parameters to compiler decisions

Parameter	Compiler Pass	Effect
BLOCK_M/N/K	Tile Formation	Tile size
num_warps	Thread Mapping	Warp decomposition
num_stages	Software Pipelining	Latency hiding
Grid	Program Scheduling	CTA generation
dtype	Instruction Selection	Tensor Core eligibility